This is a copy of a conversation between ChatGPT & Anonymous.

Report conversation

K8s YAMLs for .NET

PR Link: https://github.com/g12-4soat/tech-lanches/pull/66

Conversation Link: https://chat.openai.com/share/fdf5e03a-c4b0-498a-968c-8585714840d4

generate kubernetes yaml files for an application having the following information:

the k8s files should contain
- the applications are in .net 6
- all files should match the namespace "techlanches". a file/command to create it is a must
- pod for an api called techlanches-api (hosted in 5050 port internally & expose in port 80)
- pod for an worker (background service) called techlanches-worker. the worker is a IHostedBackground service, so include the /health endpoint to it in the configs (use port 5060)
- the images for api/worker pods are in dotnet 6, under "g12-4soat" dockerhub user and they're named techlanches-api and techlanches-worker respectively. they are public so no auth is necessary
- pod and volumes (separate pv/pvc yaml files) for a sql server 2019 database image
- hpa files for both api and worker having scale up to 3 pods (standart) and scale down to 1 pod when cpu and/or memory are under 50%
- secrets to store DefaultConnection string to database that will be used for both api/worker
- configMap in case the worker needs to communicate to the api via http requests
- generate separate deployment files for all pods
- generate separate loadbalancer service files for the api and worker
- generate probes for api and worker
- link all necessary variables (config, secrets) between these files

show the command sequences to apply all those scripts

To achieve the described setup, you need several Kubernetes YAML files. Below are the necessary files along with their content. Make sure to save each content in separate files (e.g., `techlanches-namespace.yaml`, `techlanches-api-deployment.yaml`, etc.).

1. **Namespace File (`techlanches-namespace.yaml`):**

Code
```
# techlanches-namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: techlanches
```

Apply the namespace using the following command:

Code
```
kubectl apply -f techlanches-namespace.yaml
```

2. **Secrets File (`techlanches-secrets.yaml`):**

Code
```
# techlanches-secrets.yaml
apiVersion: v1
kind: Secret
metadata:
  name: techlanches-secrets
  namespace: techlanches
type: Opaque
data:
  DefaultConnection: <base64-encoded-connection-string>
```

Encode your connection string to Base64 and replace `<base64-encoded-connection-string>` with the result.

Apply the secrets using the following command:

Code
```
kubectl apply -f techlanches-secrets.yaml
```

3. **ConfigMap File (`techlanches-configmap.yaml`):**

Code
```
# techlanches-configmap.yaml
apiVersion: v1
```

```
kind: ConfigMap
metadata:
  name: techlanches-configmap
  namespace: techlanches
data:
  api-url: "http://techlanches-api:5050"
```

Apply the config map using the following command:

Code

```
kubectl apply -f techlanches-configmap.yaml
```

4. **PersistentVolume (PV) and PersistentVolumeClaim (PVC) Files for SQL Server (`techlanches-sql-pv-pvc.yaml`):**

Code

```
# techlanches-sql-pv-pvc.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: techlanches-sql-pv
  namespace: techlanches
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/path/to/sql/data"

---

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: techlanches-sql-pvc
  namespace: techlanches
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

Modify the `path` in `hostPath` to the actual path where you want to store SQL Server data.

Apply the PV and PVC using the following command:

Code

```
kubectl apply -f techlanches-sql-pv-pvc.yaml
```

5. **Deployment Files (`techlanches-api-deployment.yaml` and `techlanches-worker-deployment.yaml`):**

Code

```
# techlanches-api-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: techlanches-api
  namespace: techlanches
spec:
  replicas: 1
  selector:
    matchLabels:
      app: techlanches-api
  template:
    metadata:
      labels:
        app: techlanches-api
    spec:
      containers:
        - name: techlanches-api
          image: g12-4soat/techlanches-api:latest
          ports:
            - containerPort: 5050
---
# techlanches-worker-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: techlanches-worker
  namespace: techlanches
spec:
  replicas: 1
  selector:
    matchLabels:
      app: techlanches-worker
  template:
    metadata:
      labels:
        app: techlanches-worker
    spec:
      containers:
        - name: techlanches-worker
          image: g12-4soat/techlanches-worker:latest
          ports:
            - containerPort: 5060
```

Apply the deployment files using the following commands:

Code

```
kubectl apply -f techlanches-api-deployment.yaml
kubectl apply -f techlanches-worker-deployment.yaml
```

6. **Service Files (`techlanches-api-service.yaml` and `techlanches-worker-service.yaml`):**

Code

```
# techlanches-api-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: techlanches-api-service
  namespace: techlanches
spec:
  selector:
    app: techlanches-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5050
  type: LoadBalancer
---
# techlanches-worker-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: techlanches-worker-service
  namespace: techlanches
spec:
  selector:
    app: techlanches-worker
  ports:
    - protocol: TCP
      port: 5060
      targetPort: 5060
  type: LoadBalancer
```

Apply the service files using the following commands:

Code

```
kubectl apply -f techlanches-api-service.yaml
kubectl apply -f techlanches-worker-service.yaml
```

7. **Horizontal Pod Autoscaler (HPA) Files (`techlanches-api-hpa.yaml` and `techlanches-worker-hpa.yaml`):**

Code

```
# techlanches-api-hpa.yaml
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: techlanches-api-hpa
  namespace: techlanches
spec:
```

```
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: techlanches-api
  minReplicas: 1
  maxReplicas: 3
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 50
---
# techlanches-worker-hpa.yaml
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: techlanches-worker-hpa
  namespace: techlanches
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: techlanches-worker
  minReplicas: 1
  maxReplicas: 3
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 50
```

Apply the HPA files using the following commands:

Code

```
kubectl apply -f techlanches-api-hpa.yaml
kubectl apply -f techlanches-worker-hpa.yaml
```

After applying these files, your Kubernetes cluster should have the necessary resources configured for your .NET 6 application.

generate a shell file that contains all kubectl apply commands to execute all at once

Sure, you can create a shell script (e.g., `apply-all.sh`) with the following content:

Code

```
#!/bin/bash

# Apply Namespace
kubectl apply -f techlanches-namespace.yaml

# Apply Secrets
kubectl apply -f techlanches-secrets.yaml

# Apply ConfigMap
kubectl apply -f techlanches-configmap.yaml

# Apply PV and PVC for SQL Server
kubectl apply -f techlanches-sql-pv-pvc.yaml

# Apply Deployments
kubectl apply -f techlanches-api-deployment.yaml
kubectl apply -f techlanches-worker-deployment.yaml

# Apply Services
kubectl apply -f techlanches-api-service.yaml
kubectl apply -f techlanches-worker-service.yaml

# Apply HPA
kubectl apply -f techlanches-api-hpa.yaml
kubectl apply -f techlanches-worker-hpa.yaml
```

Make sure to give execute permissions to the script:

Code

```
chmod +x apply-all.sh
```

Then, you can run the script:

Code

```
./apply-all.sh
```

This script will execute all the `kubectl apply` commands at once, applying the configuration to your Kubernetes cluster.

why the permissions is necessary?

Setting execute permissions on a shell script is necessary to allow the script to be run as an executable. Without execute permissions, the script cannot be executed directly. By using the `chmod +x` command, you are granting the script the permission to be executed.

Here's a breakdown:

- `chmod`: Stands for "change mode" and is used to change the file permissions.
- `+x`: Adds the execute permission to the file.

So, when you run `chmod +x apply-all.sh`, you are saying, "add execute permission to `apply-all.sh`". After setting execute permissions, you can run the script as a standalone executable by using `./apply-all.sh`.

Without execute permissions, attempting to run the script directly would result in a permission denied error.

the permission denied error could happen in any operational system? (linux, windows, mac os) or if I just run or double-click it works

The issue of permission denied typically arises in Unix-like operating systems (including Linux and macOS) when trying to execute a script. This is because these systems use file permissions to control who can execute a file. On Windows, the concept of file permissions is different, and it doesn't use the execute permission in the same way.

In Windows, you usually run scripts by specifying the interpreter (e.g., PowerShell or Command Prompt) followed by the script's path, or you can run scripts by double-clicking them if they have a recognized file extension associated with a specific interpreter.

So, if you are working on Windows and you double-click the script, it might work without needing to explicitly set execute permissions. However, if you are

working on a Unix-like system (Linux or macOS) and you try to run the script from the command line without execute permissions, you will likely encounter a permission denied error.

It's a good practice to set execute permissions explicitly when working on Unix-like systems to ensure that you can run the script from the command line. If you're primarily working on Windows, it might not be as critical, but it can still be beneficial for consistency if you ever need to share or run scripts in a different environment.

what if pv and pvc files are separeted

If your PersistentVolume (PV) and PersistentVolumeClaim (PVC) files are separated, you'll need to create the PV first and then create the PVC. Here's how you can modify the script:

1. **PersistentVolume (PV) File (`techlanches-sql-pv.yaml`):**

Code

```
# techlanches-sql-pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: techlanches-sql-pv
  namespace: techlanches
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/path/to/sql/data"
```

Apply the PV using the following command:

Code

```
kubectl apply -f techlanches-sql-pv.yaml
```

2. **PersistentVolumeClaim (PVC) File (`techlanches-sql-pvc.yaml`):**

Code

```
# techlanches-sql-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
```

```
    name: techlanches-sql-pvc
    namespace: techlanches
  spec:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 5Gi
```

Apply the PVC using the following command:

Code

```
kubectl apply -f techlanches-sql-pvc.yaml
```

With separate files, you would apply the PV first to create the volume, and then apply the PVC to claim that volume for use by your pods.

You can include these commands in your script, either before or after the Deployments and Services, depending on your specific requirements.